

Analysis Report

Like Share 875 people like this. [Sign Up](#) to see what your friends like.

[Share this Report](#)

GC log file: **gc-2017_08_16-11_30.log.0.current**

Duration: **7 min 24 sec 734 ms**

💡 Tips to reduce GC Time

(**CAUTION:** Please do thorough testing before implementing out the recommendations. These are generic recommendations & may not be applicable for your application.)

- ✓ **35.02%** of GC time (i.e 6 sec 833 ms) is caused by '**G1 Humongous Allocation**'. Humongous allocations are allocations that are larger than 50% of the region size in G1. Frequent humongous allocations can cause couple of performance issues:1. If the regions contain humongous objects, space between the last humongous object in the region and the end of the region will be unused. If there are multiple such humongous objects, this unused space can cause the heap to become fragmented.2. Until Java 1.8u40 reclamation of humongous regions were only done during full GC events. Where as in the newer JVMs, clearing humongous objects are done in cleanup phase..

Solution:

You can increase the G1 region size so that allocations would not exceed 50% limit. By default region size is calculated during startup based on the heap size. It can be overridden by specifying '-XX:G1HeapRegionSize' property. Region size must be between 1 and 32 megabytes and has to be a power of two. Note: Increasing region size is sensitive change as it will reduce the number of regions. So before increasing new region size, do thorough testing.

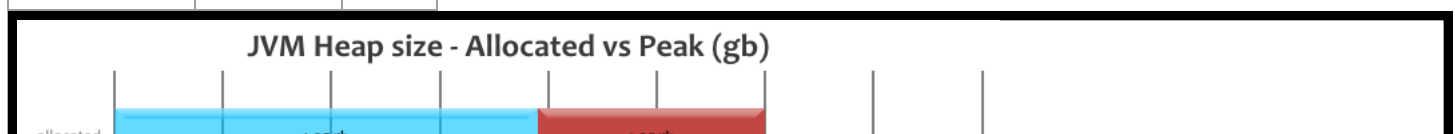
- ✓ **31.8%** of GC time (i.e 6 sec 206 ms) is caused by '**Evacuation Failure**'. When there are no more free regions to promote to the old generation or to copy to the survivor space, and the heap cannot expand since it is already at its maximum, an evacuation failure occurs. For G1 GC, an evacuation failure is very expensive - a. For successfully copied objects, G1 needs to update the references and the regions have to be tenured. b. For unsuccessfully copied objects, G1 will self-forward them and tenure the regions in place.

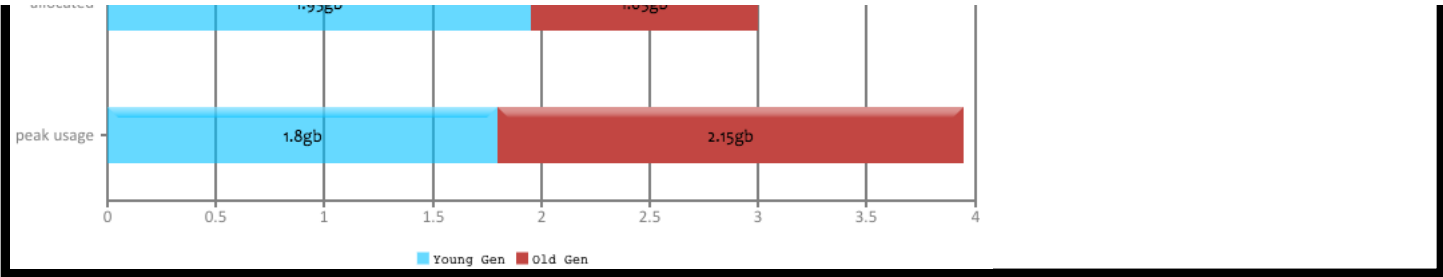
Solution:

- Evacuation failure might happen because of over tuning. So eliminate all the memory related properties and keep only min and max heap and a realistic pause time goal (i.e. Use only -Xms, -Xmx and a pause time goal -XX:MaxGCPauseMillis). Remove any additional heap sizing such as -Xmn, -XX:NewSize, -XX:MaxNewSize, -XX:SurvivorRatio, etc.
- If the problem still persists then increase JVM heap size (i.e. -Xmx)
- If you can't increase the heap size and if you notice that the marking cycle is not starting early enough to reclaim the old generation then reduce -XX:InitiatingHeapOccupancyPercent. The default value is 45%. Reducing the value will start the marking cycle earlier. On the other hand, if the marking cycle is starting early and not reclaiming, increase the -XX:InitiatingHeapOccupancyPercent threshold above the default value.
- If concurrent marking cycles are starting on time, but takes long time to finish then increase the number of concurrent marking thread count using the property: '-XX:ConcGCThreads'.
- If there are lot of 'to-space exhausted' or 'to-space overflow' GC events, then increase the -XX:G1ReservePercent. The default is 10% of the Java heap. Note: G1 GC caps this value at 50%.

☰ JVM Heap Size

Generation	Allocated ⓘ	Peak ⓘ
Young Generation	1.95 gb	1.8 gb
Old Generation	1.05 gb	2.15 gb
Total	3 gb	2.93 gb





🔍 Key Performance Indicators

(Important section of the report. To learn more about KPIs, [click here](https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/) (https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/))

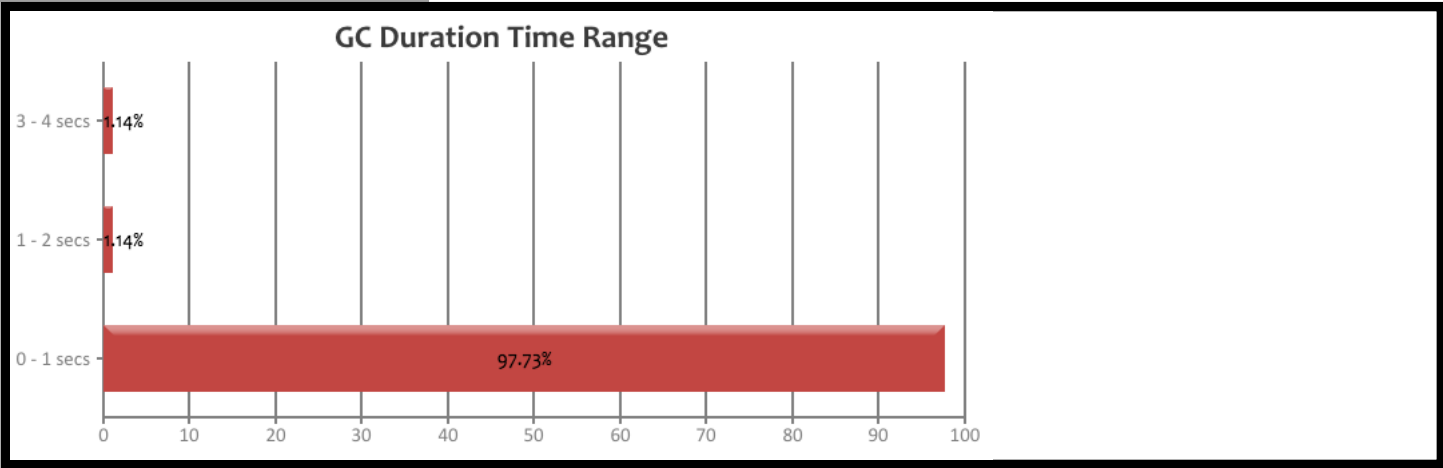
1 Throughput 📈 : 96.625%

2 Latency:

Avg GC Time 📈	146 ms
Max GC Time 📈	3 sec 690 ms

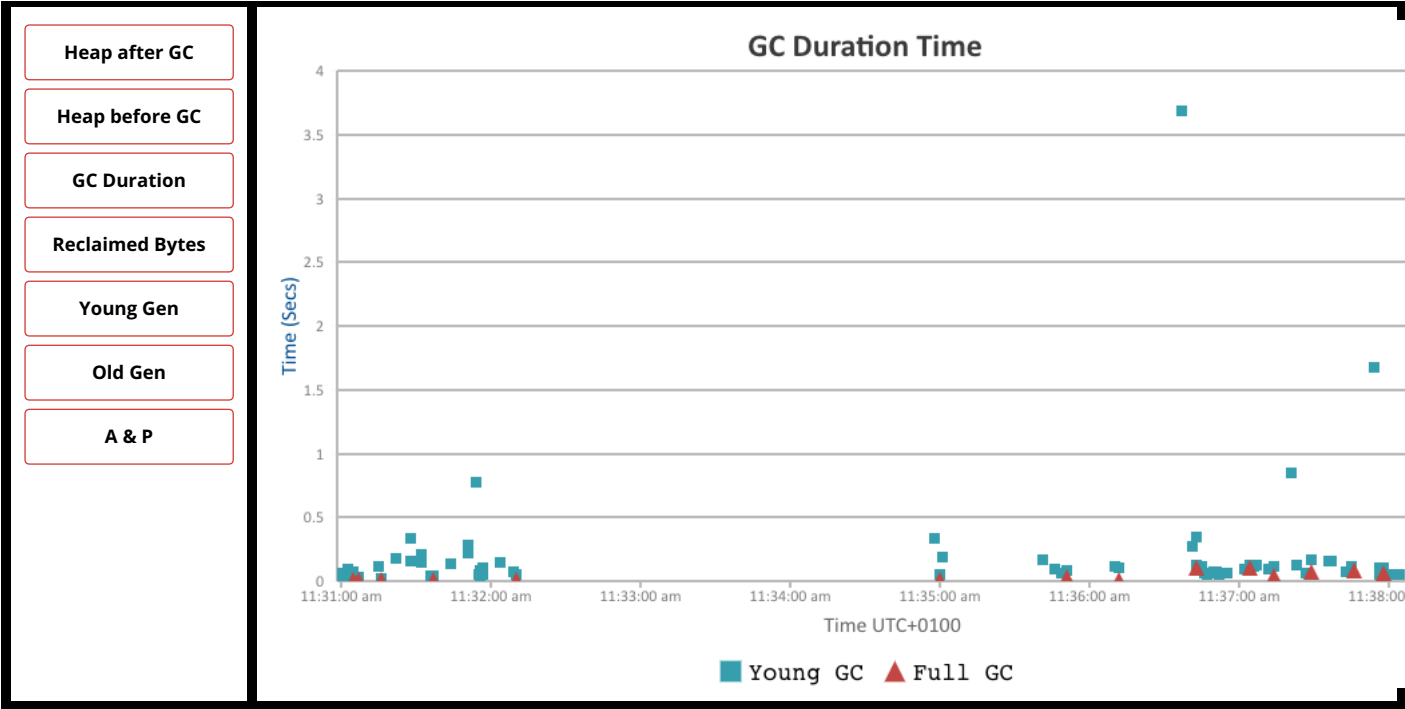
GC Duration Time Range 📈:

Duration (secs)	No. of GCs	Percentage
0 - 1	86	97.727%
1 - 2	1	98.864%
3 - 4	1	100.0%

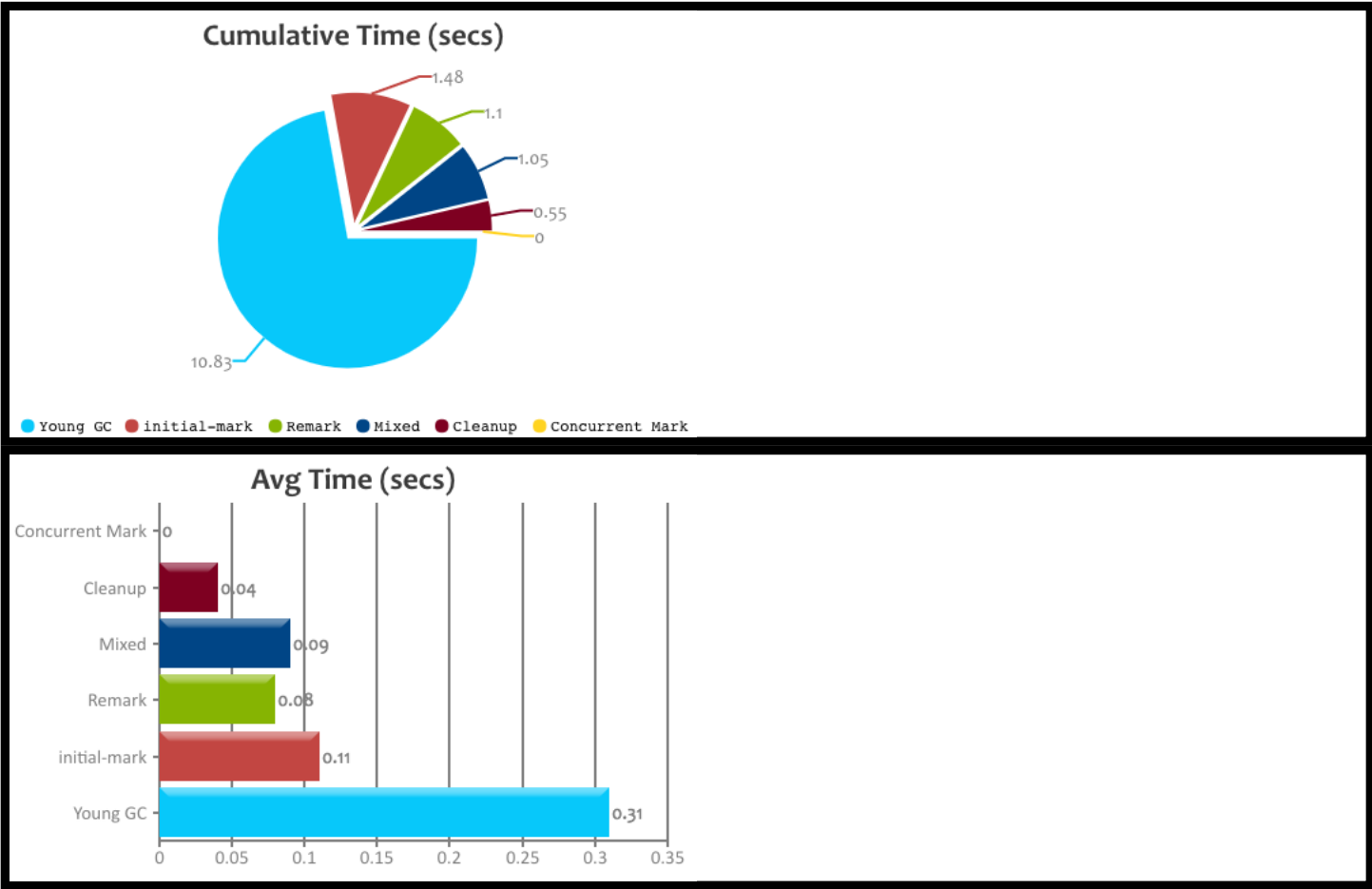


📊 Interactive Graphs

(All graphs are zoomable)



G1 GC Statistics



	Remark	initial-mark	Mixed	Young GC	Concurrent Mark	Cleanup	Total
Count ⓘ	14	14	12	35	14	14	103

Total GC Time ⓘ	1 sec 100 ms	1 sec 480 ms	1 sec 50 ms	10 sec 830 ms	0	550 ms	15 sec 10 ms
Avg GC Time ⓘ	79 ms	106 ms	88 ms	309 ms	0	39 ms	146 ms
Avg Time std dev	49 ms	86 ms	37 ms	657 ms	0	34 ms	404 ms
Min/Max Time ⓘ	0 / 170 ms	0 / 340 ms	0 / 160 ms	0 / 3 sec 690 ms	0 / 0	0 / 100 ms	0 / 3 sec 690 ms
Avg Interval Time ⓘ	31 sec 799 ms	31 sec 669 ms	7 sec 199 ms	13 sec 80 ms	31 sec 799 ms	31 sec 806 ms	22 sec 431 ms

⚙ Object Stats

(These are good metrics to include in your performance reports)

Total created bytes ⓘ	43.67 gb
Total promoted bytes ⓘ	1.53 gb
Avg creation rate ⓘ	100.56 mb/sec
Avg promotion rate ⓘ	3.53 mb/sec

💧 Memory Leak ⓘ

No major memory leaks.

(**Note:** there are [8 flavours of OutOfMemoryErrors](https://tier1app.files.wordpress.com/2014/12/outofmemoryerror2.pdf) (https://tier1app.files.wordpress.com/2014/12/outofmemoryerror2.pdf). With GC Logs you can diagnose only 5 flavours of them(Java heap space, GC overhead limit exceeded, Requested array size exceeds VM limit, Permgen space, Metaspace). So in other words, your application could be still suffering from memory leaks, but need other tools to diagnose them, not just GC Logs.)

⏴ Consecutive Full GC ⓘ

None.

⏴ Long Pause ⓘ

None.

🕒 Safe Point Duration ⓘ

(To learn more about SafePoint duration, [click here](https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/) (https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/))

Not Reported in the log.

❓ GC Causes ⓘ

(What events caused the GCs, how much time it consumed?)

Cause	Count	Avg Time	Max Time	Total Time	Time %
G1 Humongous Allocation 🚩	9	759 ms	3 sec 679 ms	6 sec 833 ms	35.02%
Evacuation Failure 🚩	3	2 sec 69 ms	3 sec 679 ms	6 sec 206 ms	31.8%
G1 Evacuation Pause 🚩	46	132 ms	766 ms	6 sec 71 ms	31.11%
Metadata GC Threshold 🚩	6	67 ms	120 ms	403 ms	2.07%
Total	64	n/a	n/a	19 sec 514 ms	100.0%



🔄 Tenuring Summary 🚩

Not reported in the log.

📄 Command Line Flags 🚩

```
-XX:G1ReservePercent=20 -XX:GCLogFileSize=10485760 -XX:InitialHeapSize=3221225472 -XX:+ManagementServer -XX:MaxGCPauseMillis=200 -
XX:MaxHeapSize=3221225472 -XX:NumberOfGCLogFiles=10 -XX:+PrintGC -XX:+PrintGCDateStamps -XX:+PrintGCDetails -XX:+PrintGCTimeStamps -
XX:+UseCompressedClassPointers -XX:+UseCompressedOops -XX:+UseG1GC -XX:+UseGCLogFileRotation
```

🏆 Become a DevOps champion in your organization

(Best practises/tools)

- ✓ Use **fastThread.io** (<https://fastthread.io/>) to analyze thread dumps, core dumps and hs_err_pid files
- ✓ Do proactive Garbage Collection analysis on all your JVMs (not just one or two) using the GC log analysis API 🚩 (<https://blog.gceasy.io/2016/06/18/garbage-collection-log-analysis-api/>)
- ✓ Always rotate the GC log files (<https://blog.gceasy.io/2016/11/15/rotating-gc-log-files/>) to prevent critical garbage collection data loss
- ✓ Use 'Pro' version (<http://gceasy.io/pricing.jsp>) for 10x fast, unlimited, secure usage

Do you like this report?